

# Tabula

Documentation technique du projet  
Architecture, ressources et fonctionnement

*Mars 2026*

**Python 3.12**

FastAPI + SQLAlchemy

**React 19**

TypeScript + Vite

**Dual-mode**

Mono / Multi-user

## Sommaire

1. Presentation du projet
2. Stack technique
3. Architecture du projet
4. Modeles de donnees
5. API REST — Endpoints
6. Services metier
7. Pipeline de traitement
8. Architecture frontend
9. Authentification et modes
10. Deploiement Docker
11. Dependances
12. Arborescence du projet

# 1. Presentation du projet

**Tabula** est un outil web moderne d'extraction de tables depuis des documents PDF. Il permet de visualiser un PDF, sélectionner visuellement des zones de tableaux, extraire les données structurées, et les exporter dans différents formats.

## Fonctionnalités principales

- **Upload de PDF** — glisser-déposer ou sélection de fichiers (max 100 Mo)
- **Visualisation PDF** — rendu client via react-pdf avec navigation par pages et miniatures
- **Détection automatique** — identification des zones de tableaux par PyMuPDF
- **Sélection manuelle** — outil de sélection rectangulaire sur le canvas PDF
- **Extraction intelligente** — double stratégie (lattice/stream) avec fallback OCR Tesseract
- **Export multi-format** — CSV, TSV, JSON, ZIP (un fichier par tableau)
- **Templates réutilisables** — sauvegarder/importer des sélections pour des PDF similaires
- **Mode dual** — mono-utilisateur (SQLite) ou multi-utilisateurs (PostgreSQL + JWT)
- **Administration** — gestion des utilisateurs, bascule de mode, partage de templates

# 2. Stack technique

## Backend

| Composant       | Technologie             | Role                                        |
|-----------------|-------------------------|---------------------------------------------|
| Framework web   | FastAPI 0.115+          | API REST asynchrone, validation Pydantic    |
| Serveur ASGI    | Uvicorn 0.32+           | Serveur de production performant            |
| ORM             | SQLAlchemy 2.0+         | Mapping objet-relationnel, support multi-DB |
| Migrations      | Alembic 1.14+           | Migrations de schema PostgreSQL             |
| Traitement PDF  | PyMuPDF (fitz) 1.24+    | Extraction de texte et de tables            |
| OCR             | Tesseract + pytesseract | Fallback pour PDF avec polices encodées     |
| Auth            | bcrypt + PyJWT          | Hachage de mots de passe + tokens JWT HS256 |
| Rate limiting   | slowapi 0.1.9+          | Protection contre les abus par IP           |
| Base de données | PostgreSQL 16 / SQLite  | Selon le mode (multi / mono)                |

## Frontend

| Composant    | Technologie               | Role                           |
|--------------|---------------------------|--------------------------------|
| Framework UI | React 19 + TypeScript 5.9 | Interface utilisateur reactive |

|              |                       |                                     |
|--------------|-----------------------|-------------------------------------|
| Build        | Vite 7.3              | Build rapide, HMR                   |
| Etat serveur | TanStack Query 5      | Cache, revalidation, mutations      |
| Etat client  | Zustand 5             | Store leger pour auth et selections |
| Rendu PDF    | react-pdf 10 (pdf.js) | Affichage client-side des PDF       |
| Styles       | Tailwind CSS 4        | Utility-first CSS                   |
| Composants   | shadcn/ui             | Composants accessibles (Radix)      |
| Routage      | React Router 7        | Navigation SPA                      |

### 3. Architecture du projet

L'application suit une architecture classique **client-serveur** avec une API REST. Le backend FastAPI sert a la fois l'API et les fichiers statiques du frontend (en production). En developpement, le frontend Vite tourne sur un port separe (5173).

```
Browser (React SPA)
|
| HTTP/REST + JWT
v
FastAPI (Uvicorn :8000)
|-- /api/setup, /api/auth, /api/admin
|-- /api/documents (upload, list, serve)
|-- /api/documents/{id}/extract, export
|-- /api/templates (CRUD, import/export)
|-- /api/jobs/{id} (polling status)
|
v
Services metier
|-- extraction_service (PyMuPDF + Tesseract OCR)
|-- pdf_service (metadata, validation texte)
|-- export_service (CSV, TSV, JSON, ZIP)
|-- storage_service (disque, SHA256)
|
v
SQLAlchemy ORM → PostgreSQL / SQLite
```

Le traitement des PDF (analyse, detection de tables) s'execute en arriere-plan dans un ThreadPoolExecutor de 4 workers. L'avancement est suivi via le modele Job (polling par le frontend).

#### Organisation du backend

- app/api/ — Routeurs FastAPI (endpoints HTTP, validation, serialisation)
- app/core/ — Logique transverse (auth, dependencies, job\_manager, worker\_pool, rate\_limit)
- app/models/ — Modeles SQLAlchemy (Document, Job, Template, User, types universels)
- app/schemas/ — Schemas Pydantic (requetes/reponses, validation)
- app/services/ — Services metier (extraction, PDF, export, storage, migration)
- app/config.py — Configuration bi-couche (Settings env + AppConfig JSON)
- app/database.py — Engine SQLAlchemy, session factory, reset dynamique

# 4. Modeles de donnees

Quatre modeles principaux constituent le schema. Les types UniversalUUID et UniversalJSON assurent la compatibilite SQLite (String/JSON) et PostgreSQL (UUID/JSONB).

## Document

| Colonne                 | Type               | Description                            |
|-------------------------|--------------------|----------------------------------------|
| id                      | UUID (PK)          | Identifiant unique                     |
| original_filename       | String(255)        | Nom du fichier uploadé                 |
| file_path               | String(500)        | Chemin sur disque                      |
| file_size               | Integer            | Taille en octets                       |
| page_count              | Integer (nullable) | Nombre de pages (après analyse)        |
| pages                   | JSON (nullable)    | Dimensions par page [{w, h, rotation}] |
| detected_tables         | JSON (nullable)    | Zones de tables auto-détectées         |
| sha256                  | String(64)         | Hash pour deduplication                |
| user_id                 | UUID FK (nullable) | Propriétaire (None en mono)            |
| created_at / updated_at | DateTime           | Horodatages                            |

## Job

| Colonne                 | Type              | Description                           |
|-------------------------|-------------------|---------------------------------------|
| id                      | UUID (PK)         | Identifiant unique                    |
| document_id             | UUID FK           | Document associé (cascade delete)     |
| status                  | String(20)        | queued   working   completed   failed |
| progress                | Integer           | Progression 0-100                     |
| message                 | String (nullable) | Message d'état                        |
| error_type              | String (nullable) | Type d'erreur (ex: no-text)           |
| created_at / updated_at | DateTime          | Horodatages                           |

## Template

| Colonne    | Type        | Description                                    |
|------------|-------------|------------------------------------------------|
| id         | UUID (PK)   | Identifiant unique                             |
| name       | String(255) | Nom du template                                |
| selections | JSON        | Tableau de selections [{page,x1,y1,x2,y2,...}] |

|                         |                    |                                   |
|-------------------------|--------------------|-----------------------------------|
| selection_count         | Integer            | Nombre de selections              |
| page_count              | Integer            | Nombre de pages couvertes         |
| user_id                 | UUID FK (nullable) | Proprietaire                      |
| is_shared               | Boolean            | Visible par tous les utilisateurs |
| created_at / updated_at | DateTime           | Horodatages                       |

## User

| Colonne       | Type               | Description           |
|---------------|--------------------|-----------------------|
| id            | UUID (PK)          | Identifiant unique    |
| username      | String(150) unique | Nom d'utilisateur     |
| password_hash | String(255)        | Hash bcrypt           |
| is_admin      | Boolean            | Droits administrateur |
| is_active     | Boolean            | Compte actif          |
| created_at    | DateTime           | Date de creation      |

## Relations

- User 1—N Document (FK user\_id, ON DELETE SET NULL)
- User 1—N Template (FK user\_id, ON DELETE SET NULL)
- Document 1—N Job (FK document\_id, CASCADE DELETE)

# 5. API REST — Endpoints

Tous les endpoints sont prefixes par `/api`. Le middleware **setup\_gate** bloque les requetes (sauf `/setup` et `/settings`) tant que la configuration initiale n'est pas terminee.

## Setup (/api/setup)

|             |                      |                                           |
|-------------|----------------------|-------------------------------------------|
| <b>GET</b>  | <code>/status</code> | Statut du setup (completed, mode)         |
| <b>POST</b> | <code>/init</code>   | Initialiser l'app (mono ou multi) [3/min] |

## Auth (/api/auth)

|             |                        |                                                             |
|-------------|------------------------|-------------------------------------------------------------|
| <b>POST</b> | <code>/login</code>    | Authentification, retourne access + refresh tokens [10/min] |
| <b>POST</b> | <code>/register</code> | Auto-inscription (si activee) [5/min]                       |
| <b>POST</b> | <code>/refresh</code>  | Renouveler l'access token [30/min]                          |

|     |     |                                  |
|-----|-----|----------------------------------|
| GET | /me | Profil de l'utilisateur connecte |
|-----|-----|----------------------------------|

## Documents (/api/documents)

|        |                |                                           |
|--------|----------------|-------------------------------------------|
| POST   | /upload        | Upload de PDF (max 100 Mo) [20/min]       |
| GET    |                | Liste paginee (filtree par user en multi) |
| GET    | /{\$id}        | Metadonnees d'un document                 |
| DELETE | /{\$id}        | Supprimer un document et ses fichiers     |
| GET    | /{\$id}/file   | Servir le PDF (react-pdf)                 |
| GET    | /{\$id}/tables | Zones de tables auto-detectees            |

## Extraction (/api/documents)

|      |                           |                                            |
|------|---------------------------|--------------------------------------------|
| POST | /{\$id}/extract           | Extraire les tables des selections         |
| POST | /{\$id}/export?format=... | Extraire et telecharger (csv/tsv/json/zip) |

## Templates (/api/templates)

|        |                |                                         |
|--------|----------------|-----------------------------------------|
| GET    |                | Liste des templates (perso + partages)  |
| POST   |                | Creer un template depuis des selections |
| GET    | /{\$id}        | Detail d'un template                    |
| PUT    | /{\$id}        | Modifier un template                    |
| DELETE | /{\$id}        | Supprimer un template                   |
| GET    | /{\$id}/export | Exporter en .tabula-template.json       |
| POST   | /import        | Importer un fichier template            |
| PUT    | /{\$id}/share  | Toggler le partage (admin)              |

## Jobs (/api/jobs)

|     |         |                                         |
|-----|---------|-----------------------------------------|
| GET | /{\$id} | Statut du job (polling par le frontend) |
|-----|---------|-----------------------------------------|

## Admin (/api/admin)

|        |                  |                                              |
|--------|------------------|----------------------------------------------|
| GET    | /users           | Lister tous les utilisateurs                 |
| POST   | /users           | Creer un utilisateur                         |
| DELETE | /users/{\$id}    | Supprimer un utilisateur                     |
| PUT    | /settings        | Modifier les parametres (allow_registration) |
| POST   | /switch-to-multi | Basculer en mode multi-utilisateurs          |
| POST   | /switch-to-mono  | Basculer en mode mono-utilisateur            |

## 6. Services metier

### `extraction_service.py` — Moteur d'extraction de tables

Coeur du systeme. Utilise PyMuPDF pour extraire les tables via deux strategies : **lattice** (detection des bordures) et **stream** (analyse du flux textuel). Le mode **guess** choisit automatiquement en comptant les lignes de la grille. En cas de texte illisible (polices custom encodees), un fallback OCR Tesseract segmente l'image en lignes/colonnes via clustering spatial.

### `pdf_service.py` — Analyse des metadonnees PDF

Extrait le nombre de pages, les dimensions et rotations de chaque page. Valide la presence de texte extractible (echec sinon avec `error_type='no-text'`).

### `export_service.py` — Export multi-format

Convertit les donnees extraites en CSV, TSV, JSON ou ZIP. Le format ZIP genere un fichier CSV par tableau par page.

### `storage_service.py` — Gestion des fichiers sur disque

Sauvegarde les PDF uploades dans `uploads/{doc_id}/document.pdf`. Calcule le SHA256 pour la deduplication. Valide les magic bytes PDF.

### `db_init_service.py` — Initialisation des bases de donnees

Cree les tables SQLite ou PostgreSQL via `Base.metadata.create_all()`. Teste la connectivite PostgreSQL avant le setup. Active les pragmas SQLite (FK, WAL).

### `migration_service.py` — Migration de donnees entre modes

Migre les documents et templates entre SQLite et PostgreSQL lors des changements de mode (mono vers multi et inversement), en preservant les relations utilisateur.

---

## 7. Pipeline de traitement

Le traitement d'un PDF suit un pipeline en 7 etapes, de l'upload a l'export des donnees.

**1. Upload** — L'utilisateur uploades un ou plusieurs fichiers PDF. Le backend valide les magic bytes, verifie la taille (<100 Mo), cree un enregistrement Document et un Job (`status=queued`), sauvegarde le fichier sur disque, et soumet un job au `ThreadPoolExecutor`.

**2. Analyse en arriere-plan** — Le `job_manager` extrait les metadonnees (pages, dimensions), valide la presence de texte, puis lance la detection automatique des zones de tableaux. Le Job est mis a jour (`progress`, `status`).

**3. Visualisation du PDF** — Le frontend charge le PDF via react-pdf et affiche les zones auto-detectees en superposition. L'utilisateur navigue entre les pages.

**4. Selection des zones** — L'utilisateur dessine des rectangles sur le PDF pour definir les zones de tableaux a extraire. Chaque selection a une methode d'extraction (guess, stream, lattice).

**5. Extraction** — Le backend ouvre le PDF, decoupe chaque zone selectionnee, applique la strategie d'extraction (PyMuPDF find\_tables). Si le texte est illisible (garbled), bascule vers l'OCR Tesseract.

**6. Apercu des donnees** — Les donnees extraites sont affichees dans un tableau interactif (DataPreview). L'utilisateur peut verifier la qualite avant export.

**7. Export** — L'utilisateur choisit le format (CSV, TSV, JSON, ZIP) et telecharge les donnees. Un template peut etre sauvegarde pour reutiliser les memes selections sur d'autres PDF.

### Detection de texte illisible (garbled)

Certains PDF utilisent des polices avec des encodages custom qui produisent du texte illisible lors de l'extraction. La fonction `_is_garbled(text)` detecte ce cas en verifiant que moins de 40% des caracteres sont normaux (lettres, chiffres, ponctuation). Le fallback OCR rend la zone en image 300 DPI, execute Tesseract en mode sparse-text, puis reconstruit le tableau par clustering spatial des mots.

## 8. Architecture frontend

### Pages

| Route      | Page          | Description                                      |
|------------|---------------|--------------------------------------------------|
| /setup     | SetupPage     | Assistant de configuration initiale (mono/multi) |
| /login     | LoginPage     | Formulaire de connexion (multi uniquement)       |
| /register  | RegisterPage  | Formulaire d'inscription (si active)             |
| /          | HomePage      | Bibliotheque de documents, upload                |
| /pdf/:id   | PdfViewPage   | Visualiseur PDF + extraction + export            |
| /templates | TemplatesPage | Gestion des templates (CRUD, import/export)      |
| /settings  | SettingsPage  | Parametres, admin, bascule de mode               |
| /about     | AboutPage     | A propos du projet                               |
| /help      | HelpPage      | Aide et documentation                            |



## Composants cles

- **PdfViewer** — Affichage du PDF avec react-pdf, zoom, rotation
- **SelectionOverlay** — Couche de dessin rectangulaire sur le PDF
- **ThumbnailSidebar** — Navigation par miniatures de pages
- **ControlPanel** — Controles de zoom, page, rotation
- **DataPreview** — Tableau interactif des donnees extraites
- **ExportControls** — Boutons d'export (CSV, TSV, JSON, ZIP)
- **MethodSelector** — Choix de la methode d'extraction
- **FileUploader** — Zone de drag-and-drop pour l'upload
- **DocumentLibrary** — Liste des documents avec actions
- **TemplateLibrary** — Liste des templates avec actions
- **AuthGuard** — Protection des routes en mode multi
- **Navbar** — Barre de navigation + menu utilisateur

## Gestion de l'etat

| Store                | Donnees                           | Persistance                  |
|----------------------|-----------------------------------|------------------------------|
| Zustand (auth-store) | Tokens JWT, user, isAuthenticated | Persiste en localStorage     |
| Zustand (store)      | Selections rectangulaires         | Etat ephemere (session)      |
| TanStack Query       | Documents, templates, jobs        | Cache serveur, staleTime 30s |

## Client HTTP

Le module `lib/api.ts` centralise toutes les requetes HTTP. Il ajoute automatiquement le header `Authorization: Bearer`, gere le renouvellement transparent du token en cas de 401, et redirige vers `/setup` en cas de 503 (setup requis).

# 9. Authentification et modes

## Mode mono-utilisateur

En mode mono, aucune authentification n'est requise. La dependance `get_current_user()` retourne `None`, et tous les endpoints sont accessibles. La base de donnees est un fichier SQLite local.

## Mode multi-utilisateurs

En mode multi, chaque requete doit inclure un JWT valide. Le frontend stocke les tokens en `localStorage` et les renouvelle automatiquement.

| Etape | Action         | Detail                                                    |
|-------|----------------|-----------------------------------------------------------|
| 1     | Login          | POST /auth/login avec username + password                 |
| 2     | Verification   | bcrypt compare le hash en base                            |
| 3     | Tokens         | Retourne access_token (60 min) + refresh_token (30 jours) |
| 4     | Requetes       | Header Authorization: Bearer {access_token}               |
| 5     | Renouvellement | Sur 401, tente un refresh automatique (1 fois)            |
| 6     | Expiration     | Si refresh echoue, redirection vers /login                |

## Isolation des donnees

- Chaque utilisateur ne voit que ses propres documents et templates
- Les administrateurs voient et gerent toutes les ressources
- Les templates partages (is\_shared=True) sont visibles en lecture seule par tous
- Seul un admin peut toggler le partage d'un template

## Bascule de mode

L'application peut basculer entre les modes a chaud via les endpoints admin. La bascule mono vers multi requiert une connexion PostgreSQL et la creation d'un admin. La bascule multi vers mono cree un nouveau SQLite et peut migrer les donnees d'un utilisateur.

# 10. Deploiement Docker

## Dockerfile (multi-stage)

- **Stage 1 : frontend-builder** — Node 22-slim, npm install, npm run build → dist/
- **Stage 2 : runtime** — Python 3.12-slim, Tesseract OCR, uv (deps), backend + static frontend

## docker-compose.yml

| Service | Image               | Ports     | Volumes           |
|---------|---------------------|-----------|-------------------|
| db      | PostgreSQL 16       | 5433:5432 | pgdata            |
| app     | Tabula (Dockerfile) | 8000:8000 | appdata + uploads |

## Script d'entrypoint

Le `docker-entrypoint.sh` execute les migrations Alembic uniquement si le fichier de configuration existe et que le mode est 'multi' (PostgreSQL). Sinon, le serveur démarre directement et le setup wizard gère l'initialisation.

## Variables d'environnement

| Variable     | Defaut                    | Description                                |
|--------------|---------------------------|--------------------------------------------|
| DATA_DIR     | ./backend/data            | Repertoire de la configuration persistante |
| UPLOAD_DIR   | ./uploads                 | Repertoire de stockage des PDF             |
| DEBUG        | true                      | Mode debug (logging detaillé)              |
| CORS_ORIGINS | ["http://localhost:5173"] | Origines CORS autorisées                   |

# 11. Dependances

## Backend (Python)

| Package           | Version   | Usage                 |
|-------------------|-----------|-----------------------|
| fastapi           | >=0.115.0 | Framework web         |
| uvicorn[standard] | >=0.32.0  | Serveur ASGI          |
| sqlalchemy        | >=2.0.0   | ORM                   |
| alembic           | >=1.14.0  | Migrations DB         |
| psycopg2-binary   | >=2.9.0   | Driver PostgreSQL     |
| pydantic-settings | >=2.6.0   | Configuration         |
| PyMuPDF           | >=1.24.0  | Traitement PDF        |
| pytesseract       | >=0.3.10  | OCR Tesseract         |
| Pillow            | >=10.0.0  | Traitement d'images   |
| bcrypt            | >=4.0.0   | Hachage mots de passe |
| PyJWT             | >=2.8.0   | Tokens JWT            |
| slowapi           | >=0.1.9   | Rate limiting         |
| python-multipart  | >=0.0.12  | Upload de fichiers    |

## Frontend (Node.js)

| Package | Version | Usage |
|---------|---------|-------|
|---------|---------|-------|

|                       |          |                       |
|-----------------------|----------|-----------------------|
| react                 | ^19.2.0  | Framework UI          |
| react-router-dom      | ^7.13.1  | Routage SPA           |
| react-pdf             | ^10.4.1  | Rendu PDF (pdf.js)    |
| zustand               | ^5.0.11  | Gestion d'etat client |
| @tanstack/react-query | ^5.90.21 | Cache serveur         |
| @tailwindcss/vite     | ^4.2.1   | Styles utilitaires    |
| sonner                | ^2.0.7   | Notifications toast   |
| lucide-react          | ^0.525.0 | Icones                |
| vite                  | ^7.3.1   | Build tool            |
| typescript            | ~5.9.3   | Typage statique       |
| vitest                | ^4.0.18  | Tests unitaires       |

## 12. Arborescence du projet

```

tabula/
+-- backend/
|   +-- app/
|   |   +-- main.py
|   |   +-- config.py
|   |   +-- database.py
|   |   +-- api/           # Routeurs (setup, auth, admin, documents, extraction, templates,
jobs)
|   |   +-- core/         # Auth, dependencies, job_manager, worker_pool, rate_limit
|   |   +-- models/      # Document, Job, Template, User, types universels
|   |   +-- schemas/     # Schemas Pydantic (requetes/reponses)
|   |   +-- services/    # extraction, pdf, export, storage, db_init, migration
|   +-- alembic/         # Migrations de schema
|   +-- tests/           # 105 tests (pytest)
|   +-- requirements.txt
+-- frontend/
|   +-- src/
|   |   +-- App.tsx
|   |   +-- pages/       # 9 pages (Home, PdfView, Templates, Settings, ...)
|   |   +-- components/  # pdf-viewer, extraction, auth, layout, documents, ui
|   |   +-- hooks/       # useDocuments, useExtraction, useTemplates, ...
|   |   +-- lib/         # api.ts, types.ts, auth-store.ts, store.ts
|   +-- package.json
|   +-- vite.config.ts
+-- docker-compose.yml
+-- Dockerfile
+-- docker-entrypoint.sh
+-- .env.example
+-- README.md

```